

Common Functions in Data Science Libraries

NumPy – Numerical Computing Essentials

NumPy is the fundamental package for numerical arrays and mathematical operations in Python. It provides the `ndarray` structure for efficient storage and computation on homogeneous data. Commonly used NumPy functions include creating arrays, generating sequences of numbers, reshaping arrays, and performing aggregations or linear algebra operations:

- **Array creation:** `np.array(obj)` converts a list (or other sequence) into a NumPy array, which is a powerful data structure for large numeric datasets ¹. For example, `np.arange(start, stop, step)` generates an array of evenly spaced values over a range ², and `np.linspace(start, stop, num)` creates a specified number of values evenly spaced between two endpoints ².
- **Reshaping and combining:** `np.reshape(a, new_shape)` gives a new shape to an array without changing its data ³, and `np.transpose(a)` swaps the axes of an array (e.g. converting rows to columns) ⁴. To join arrays, `np.concatenate([a, b], axis=0)` merges multiple arrays into one along a given axis ⁵. Similarly, `np.unique(a)` finds the unique elements of an array ⁶.
- **Random number generation:** The `numpy.random` module provides functions like `np.random.rand(n, m)` to generate an array of shape (n, m) with random values in [0,1) ⁷ (other functions exist for different distributions). This is useful for simulations or initializing random values.
- **Aggregation and statistics:** NumPy makes it easy to compute summary statistics across arrays. For instance, `np.sum(a)` returns the sum of all elements in `a` ⁸. Functions like `np.mean(a)`, `np.median(a)`, `np.std(a)` compute the mean, median, and standard deviation of an array respectively (optionally along a specific axis) ⁹ ¹⁰. You can also obtain the min/max values with `np.min(a)` or `np.max(a)`. These vectorized operations are highly optimized.
- **Linear algebra:** NumPy provides efficient linear algebra routines. For example, `np.dot(A, B)` computes the dot product of two arrays (for 2D arrays, this is matrix multiplication) ⁴. In recent NumPy versions, the `@` operator can also be used for matrix multiplication. There are also functions for computing norms, solving linear systems, eigenvalues, etc., in the submodule `numpy.linalg`.

pandas – Data Manipulation and Analysis

pandas is a library built on NumPy that offers high-level data structures (especially the `DataFrame`) and functions designed to make data analysis easy and expressive. A pandas **DataFrame** is like a table with labeled rows and columns, suitable for heterogeneous tabular data. The following functions are frequently used when working with pandas DataFrames:

- **Loading data:** `pd.read_csv('file.csv')` reads a CSV file into a `DataFrame` ¹¹ (similarly, there are `pd.read_excel`, `pd.read_json`, etc. for other formats). This is often the first step in any data analysis pipeline.

- **Viewing and inspecting data:** After loading, you can use `df.head(n)` to preview the first n rows of the DataFrame (by default $n=5$) ¹², and `df.tail(n)` for the last n rows. `df.info()` displays a concise summary of the DataFrame, including each column's data type and the count of non-missing values ¹³. For numeric columns, `df.describe()` generates summary statistics (count, mean, std, min, quartiles, max) ¹⁴. Additionally, `df.shape` gives the dimensions of the DataFrame (number of rows, number of columns) ¹⁵, which is useful to quickly verify data size.
- **Indexing and selection:** You can select columns by name (e.g. `df['col_name']`) or rows by index. pandas provides label-based indexing via `df.loc[...]` and position-based indexing via `df.iloc[...]`. For example, `df.loc[0]` gives the first row, and `df.iloc[:5, :3]` gives the first 5 rows and first 3 columns. These are used to filter or subset the data.
- **Handling missing data:** pandas has convenient methods to detect and handle missing values. `df.isnull()` returns a DataFrame of booleans indicating missing entries ¹⁶. To drop missing data, `df.dropna()` removes any rows with **NaN** (by default) ¹⁷. Conversely, `df.fillna(value)` replaces NaNs with a specified *value* (for instance, 0 or the mean of the column) ¹⁸. These functions have variants or parameters to drop columns instead, or to only drop rows if *all* values are missing, etc.
- **Value counts and unique values:** For categorical data or any column, `df['col'].value_counts()` is very useful – it returns the frequency count of each unique value in that column ¹⁹. Meanwhile, `df['col'].unique()` gives the array of unique values. These help in understanding the distribution of categorical features.
- **Grouping and aggregation:** A powerful feature of pandas is grouping data and aggregating. Using `df.groupby('col')` splits the DataFrame into groups based on unique values in *col*. You can then apply aggregation functions on each group. For example, `df.groupby('category')['value'].mean()` computes the mean of the *value* column for each category. More generally, `df.groupby('col').agg({'val1': 'mean', 'val2': 'sum'})` can compute multiple aggregations at once, like mean of *val1* and sum of *val2* per group ²⁰. This is extremely handy for summarizing data by categories.
- **Merging and joining:** pandas DataFrames can be merged (SQL join style) by using `pd.merge`. For example, `pd.merge(df1, df2, on='key')` will join two DataFrames on a common column named “key” ²¹. You can specify different join types (inner, left, right, outer) and different keys for each side. Additionally, you can concatenate DataFrames with `pd.concat`. For instance, `pd.concat([df1, df2], axis=0)` will stack two DataFrames vertically (one below the other), and `axis=1` would stack them horizontally (adding more columns) ²².
- **Sorting:** Use `df.sort_values(by='col', ascending=True)` to sort the DataFrame by one or more columns ²³. This is often needed to find top values or to organize output. You can also sort by the index with `df.sort_index()`.
- **Index management:** You can change the index of the DataFrame with `df.set_index('column_name')`, which will make that column into the row index ²⁴. Conversely, `df.reset_index()` will move the index back to a normal column (and create a new default integer index) ²⁵. Managing the index is useful for time series (using dates as an index, for example) or when the index has meaning (like unique identifiers).
- **Pivoting:** Pandas can pivot data to reshape it. `pd.pivot_table(df, values='val', index='group_col', columns='category_col', aggfunc='mean')` will create a pivot table: it groups data by *group_col* and *category_col*, then computes the mean of *val* for each combination ²⁶. The result is a table with one index per group and one column per category, filled with the aggregated values. This is useful for cross-tabulations and summary reports.

- **Applying functions:** You can apply arbitrary functions to your data. For example, `df['col'].apply(func)` applies a function *func* to every element of the column ²⁷. There's also `df.apply(func, axis=1)` to apply a function across each row. This is a flexible way to perform custom transformations that aren't built into pandas. For more complex elementwise operations, using vectorized NumPy functions is recommended for performance, but `apply` is convenient for custom logic.

Finally, once you've transformed your data, you may want to **export** it. Functions like `df.to_csv('output.csv', index=False)` will write the DataFrame to a CSV file ²⁸ (with `index=False` to omit the row index). Similarly, there are `to_excel`, `to_json`, etc. for other formats.

SciPy (Statistical Functions and More)

SciPy is a library that builds on NumPy and provides a large collection of scientific functions and algorithms. Of particular interest is `scipy.stats`, which contains a wide range of statistical distributions and tests. According to the SciPy documentation, this module contains *"a large number of probability distributions, summary and frequency statistics, correlation functions and statistical tests"* ²⁹. In data analysis tasks, you might use SciPy to perform statistical tests or work with statistical distributions. Some general functions with high probability of appearing include:

- **T-tests:** SciPy can perform *t*-tests to compare means. For example, `stats.ttest_ind(sample1, sample2)` conducts a **two-sample t-test** for the null hypothesis that two independent samples have the same average ³⁰ ³¹. It returns the t-statistic and p-value ³², allowing you to assess if the difference in means is statistically significant. Similarly, `stats.ttest_1samp(sample, mu)` tests if the mean of a single sample differs from a given value (one-sample t-test), and `stats.ttest_rel(sample1, sample2)` is used for *paired* t-tests on dependent samples (e.g. before/after measurements).
- **ANOVA (Analysis of Variance):** For comparing means across more than two groups, `stats.f_oneway(group1, group2, ...)` performs a one-way ANOVA test ³³. It returns the F-statistic and p-value to indicate if at least one group mean is different. If you have related measures or need non-parametric tests, SciPy also offers alternatives (like `stats.kruskal` for a non-parametric ANOVA and `stats.friedmanchisquare` for repeated measures).
- **Chi-squared test:** `stats.chisquare(f_obs, f_exp)` performs Pearson's **chi-square test** for goodness-of-fit ³⁴. This test checks whether an observed frequency distribution (*f_obs*) differs from an expected distribution (*f_exp*). For example, if you have observed counts of categories and expected proportions, `stats.chisquare` can tell you if the deviations are likely due to chance. The SciPy docs describe it as a test that *"assesses the null hypothesis that the observed frequencies... are obtained by independent sampling from a categorical distribution with given expected frequencies."* ³⁵. The result includes the chi-square statistic and a p-value.
- **Correlation:** To measure linear relationship between two variables, `stats.pearsonr(x, y)` computes the **Pearson correlation coefficient** *r* and a p-value for testing non-correlation ³⁶. The coefficient ranges from -1 to 1, indicating perfectly negative or positive linear relationships at the extremes, with 0 meaning no linear correlation. The accompanying p-value tells you if the correlation is significantly different from zero (under the assumption of bivariate normal data) ³⁷. SciPy also provides `stats.spearmanr` for Spearman rank correlation (non-parametric) and `stats.kendalltau` for Kendall's tau.

- **Distribution functions:** SciPy's `stats` module includes objects for many probability distributions (normal, binomial, Poisson, etc.). For example, `stats.norm` represents a normal (Gaussian) distribution. You can use methods like `stats.norm.cdf(x)` for the cumulative distribution function, `stats.norm.pdf(x)` for the probability density function, or `stats.norm.rvs(size=n)` to generate n random samples from a standard normal distribution. Likewise, `stats.expon`, `stats.binom`, and many others exist. This is useful for computing p-values or critical values directly from distributions (e.g., `stats.norm.ppf(0.975)` gives the 97.5th percentile of the standard normal).
- **Other statistical tests:** SciPy implements a variety of hypothesis tests, such as `stats.mannwhitneyu(a, b)` for the Mann-Whitney U test (comparing two distributions non-parametrically), `stats.wilcoxon(a, b)` for the Wilcoxon signed-rank test (paired non-parametric test), `stats.bartlett` or `stats.levene` for testing equality of variances, and more. It also offers linear regression (`stats.linregress` gives a quick linear fit and correlation) and contingency table analyses (`stats.chi2_contingency` for independence tests in a contingency table). While these are less likely to all appear in a single context, it's good to be aware that SciPy covers many "one-off" statistical analysis needs.

In summary, **SciPy** is often used when you need to perform a statistical test or use a special function (like computing an integral, optimization, Fourier transform, etc.) that goes beyond the basics provided by NumPy. Its `stats` submodule is particularly rich in functions for statistical analysis, complementing higher-level libraries like statsmodels.

statsmodels – Statistical Modeling and Regression Analysis

statsmodels is a library that provides classes and functions for statistical modeling—especially regression analysis—with detailed output. It allows the use of R-style **formulas** to specify models, making it very user-friendly for statistical tests and analyses (under the hood it uses `patsy` to parse formulas). Given that we have `statsmodels` (and its dependency `patsy`) available, a common task is to fit an Ordinary Least Squares (OLS) linear regression and examine the results. Key functions and steps include:

- **Defining a model with a formula:** Using the `statsmodels.formula.api` (often imported as `smf`), you can create a regression model with a formula string. For example: `smf.ols("Y ~ X1 + X2", data=df)` specifies a linear model where Y is the dependent variable and $X1$, $X2$ are independent variables (predictors)³⁸. This formula notation is very convenient – it automatically handles adding an intercept and can include factors like categorical variables (you can wrap a variable in `C()` to treat it as categorical). The output of `smf.ols(...)` is a model object which is not yet fitted to data.
- **Fitting the model:** Once the model is defined, call `.fit()` on it to estimate the parameters (perform the regression). For example: `result = smf.ols("Y ~ X1 + X2", data=df).fit()`. This returns a results object (often called `result` or `res`) which contains the fitted parameters, residuals, etc. Under the hood, OLS fitting means it finds the coefficients that minimize the sum of squared residuals.
- **Examining results:** The results object has many attributes and methods. A particularly useful one is `result.summary()`, which prints a detailed report of the regression results³⁹. This summary includes the model information (like R-squared, F-statistic, etc.) and a table of coefficients with their estimates, standard errors, t-statistics, and p-values⁴⁰. For instance, you can see which predictors are statistically significant (p-value < 0.05) in explaining the response variable. The summary also

shows confidence intervals for coefficients and diagnostic measures like the Durbin-Watson statistic for autocorrelation of residuals ⁴¹. This rich output is one reason to use statsmodels for statistical analysis instead of scikit-learn, which by design focuses more on prediction and less on detailed inference.

- **Predicting and other methods:** With a fitted model, you can use `result.predict(new_data)` to generate predictions for new inputs (where *new_data* is a DataFrame of predictors). statsmodels also provides influence and outlier measures, hypothesis testing on coefficients (e.g., `result.pvalues` for each coefficient's p-value), and functions for confidence intervals (`result.conf_int()`). If you included categorical variables, statsmodels takes care of one-hot encoding them in the design matrix automatically (unless you specified contrast differently).
- **Other models:** While OLS is very common, statsmodels can also fit other regression models. For example, `smf.logit("Y ~ X1 + X2", data=df)` for logistic regression (when Y is binary), or use `statsmodels.api` to create a `Logit` or `GLM` (generalized linear model) with different distributions. It also has ARIMA for time series, discrete choice models, etc. But if a problem is expecting a regression output, OLS via `.ols` is a prime candidate. In our context, since scikit-learn's `LogisticRegression` is available for classification, statsmodels might primarily be used for linear regression or when we want detailed statistics.

In summary, **statsmodels** is likely to come up when we need **regression analysis with interpretability**. You'd write a formula for the model, fit it, and then use the summary or attributes to interpret how the predictors relate to the outcome. For instance, one might ask: "Using statsmodels, fit a linear model to see if there's a significant difference between groups after controlling for some variable," etc., which would entail looking at coefficient p-values in the summary.

scikit-learn – Machine Learning Utilities

scikit-learn is a very popular machine learning library that provides efficient implementations of many algorithms for classification, regression, clustering, etc., as well as tools for data preprocessing, model selection, and evaluation. Given the imports listed (e.g., `LogisticRegression`, `TfidfVectorizer`, `TruncatedSVD`, `train_test_split`, `accuracy_score`), we anticipate tasks related to machine learning pipeline components: splitting data, transforming features, training models, and evaluating them. Common scikit-learn functions and classes likely to appear are:

- **Train-test splitting:** A fundamental step before model training is to split the dataset into a training set and a testing set. The function `train_test_split(X, y, test_size=..., random_state=...)` automates this, shuffling and splitting arrays or DataFrames into two subsets ⁴². For example, `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)` will reserve 20% of the data for testing. This ensures we can evaluate the model on unseen data.
- **Feature extraction (TF-IDF):** If text data is involved, `TfidfVectorizer` from `sklearn.feature_extraction.text` is commonly used to convert text into numerical features. `TfidfVectorizer` **"convert[s] a collection of raw documents to a matrix of TF-IDF features"**, essentially computing word frequencies with inverse-document-frequency weighting ⁴³. Typical usage: `vect = TfidfVectorizer(max_features=100, stop_words='english')` followed by `X_feats = vect.fit_transform(text_data)` ⁴⁴. This produces a sparse matrix where each column corresponds to a term (out of the top 100 terms, in this example) and each row corresponds

to a document, with values being the TF-IDF score. This is likely to appear if there's any NLP or text classification task.

- **Dimensionality reduction:** With high-dimensional data (like text TF-IDF matrices), one may use `TruncatedSVD` for dimensionality reduction. `TruncatedSVD(n_components=k)` finds the top k singular components of a matrix (often used for Latent Semantic Analysis on text data) ⁴⁵. Unlike PCA, it doesn't require centering and thus works well with sparse matrices ⁴⁶. For example, after obtaining `X_feats` from a TF-IDF vectorizer, one might do `svd = TruncatedSVD(n_components=50); X_reduced = svd.fit_transform(X_feats)` to reduce the feature space while preserving variance. This can make modeling more tractable by eliminating noise and correlations in features.
- **Model training (Logistic Regression):** Scikit-learn's API for model training is consistent: create an estimator object, call `.fit(X_train, y_train)`, then predict. Given the import, **Logistic Regression** is a classifier often used for binary classification (and multiclass by extension). For instance, `model = LogisticRegression(max_iter=1000)` creates a logistic regression model. You then call `model.fit(X_train, y_train)` to train it ⁴⁷. As W3Schools notes, "We use the `LogisticRegression()` method to create a logistic regression object. This object has a method called `fit()` that takes the independent and dependent data" for training ⁴⁸. Once trained, `model.predict(X_test)` produces predictions for the test set. You might also use `model.predict_proba(X_test)` to get predicted probabilities. LogisticRegression in scikit-learn by default applies regularization, so sometimes one sets `penalty='none'` or adjusts `C` if needed, but the default is usually fine for baseline modeling.
- **Model evaluation (Accuracy and metrics):** For classification, a straightforward metric is **accuracy** – the fraction of correct predictions. Scikit-learn provides `accuracy_score(y_true, y_pred)` which returns the accuracy proportion (or number of correct predictions if `normalize=False`) ⁴⁹. ⁵⁰ For example, if `y_test` are true labels and `y_pred` are predictions from the model, `accuracy_score(y_test, y_pred)` gives the model's accuracy on the test set. Accuracy is easy to interpret, though in cases of class imbalance we might consider other metrics (precision, recall, F1 – also available via `sklearn.metrics`). If this is a multi-class classification, `accuracy_score` still works (it computes the overall percentage of correct labels).
- **Additional scikit-learn utilities:** We might also encounter:
 - **Vectorizer vocabulary:** after fitting a `TfidfVectorizer`, the attribute `vect.get_feature_names_out()` provides the list of feature terms (words). This can be used to interpret which words are important (for example, looking at highest coefficients in logistic regression).
 - **Model coefficients:** For linear models like LogisticRegression, `model.coef_` contains the learned coefficients for each feature, and `model.intercept_` the bias term. These can be inspected to see which features positively or negatively contribute to the prediction.
 - **Cross-validation:** In some contexts, `cross_val_score(estimator, X, y, cv=5)` might be used to evaluate a model with 5-fold cross-validation, giving an array of accuracy (or other metric) scores on different folds. We know `cross_val_score` was imported in a past exam year. This provides a more robust estimate of model performance than a single train-test split.
 - **Grid search:** Similarly, `GridSearchCV` could appear if hyperparameter tuning is needed, but unless explicitly asked, an exam might not go deep into that due to time constraints.
 - **Pipelines:** scikit-learn allows chaining steps (like vectorization + dimensionality reduction + model) using `Pipeline`. If many steps are involved, sometimes they set up a pipeline for convenience. But if code is provided in pieces, one can manually do: fit transform -> feed into model, etc.

Overall, with scikit-learn, the emphasis is on the **machine learning workflow**: preprocess (e.g. create features from raw data), split data, train model, and evaluate with metrics. Expect tasks like “Train a logistic regression classifier on this dataset and report the accuracy” or “Vectorize this text data and use SVD to reduce dimensionality before classification”. The functions above are central to accomplishing those tasks.

NetworkX – Graph Analysis Functions

NetworkX is a library for creating, manipulating, and analyzing graphs (networks). With `networkx` in the environment, tasks may involve network analysis such as building a graph from data, computing connectivity or centrality measures, etc. Important functions and usage patterns include:

- **Creating a graph:** You can initialize an empty graph with `G = nx.Graph()` for an undirected graph ⁵¹ (or `nx.DiGraph()` for directed). A graph in NetworkX is a collection of nodes and edges. Nodes can be any hashable object (commonly integers or strings).
- **Adding nodes and edges:** To build the graph, use `G.add_node(node)` to add a single node, or `G.add_nodes_from([list_of_nodes])` for multiple ⁵². Similarly, `G.add_edge(u, v)` adds an edge between nodes `u` and `v` ⁵³. You can add edges in bulk with `G.add_edges_from([(u1, v1), (u2, v2), ...])`. NetworkX will implicitly add any nodes that are mentioned in edges if they weren't already in the graph. After constructing a graph, you might use `G.number_of_nodes()` and `G.number_of_edges()` to verify its size ⁵⁴.
- **Basic graph properties:** You can retrieve nodes as `G.nodes` (or `list(G.nodes)` for a list of node labels) and edges as `G.edges` ⁵⁵. You can check the degree of a node (number of edges connected) via `G.degree[node]` or get a dictionary of all degrees with `dict(G.degree())`. These basics help in exploratory analysis, like finding isolated nodes (`degree 0`) or the overall degree distribution.
- **Connectivity:** One common analysis is to find connected components of the graph. `nx.is_connected(G)` returns True if the graph is fully connected (only for undirected graphs) ⁵⁶. If the graph is not fully connected, `nx.connected_components(G)` generates sets of nodes, each set being a connected component ⁵⁷. For example, you might find the largest component with `max(nx.connected_components(G), key=len)`. In directed graphs, you'd use `nx.weakly_connected_components` or `nx.strongly_connected_components` as appropriate. These functions help identify clusters of nodes or whether the graph splits into multiple sub-networks.
- **Paths and distances:** NetworkX provides shortest path algorithms. For instance, `nx.shortest_path(G, source, target)` returns a list of nodes representing the shortest path between the given source and target. There's also `nx.shortest_path_length(G, source, target)` for the distance (number of edges) on that shortest path. If you need all-pairs shortest paths, `nx.all_pairs_shortest_path(G)` gives a dictionary of shortest paths for each source node ⁵⁸. In weighted graphs, these functions accept a `weight` parameter to account for edge weights.
- **Graph algorithms and metrics:** NetworkX implements many classic graph algorithms:
- **Centrality measures:** These identify important or influential nodes. For example, `nx.degree_centrality(G)` computes the degree centrality for each node (basically degree normalized by maximum possible degree). `nx.betweenness_centrality(G)` gives the betweenness centrality, which counts how often a node lies on shortest paths between other nodes (higher means more “broker” influence). `nx.closeness_centrality(G)` measures how close a

node is to others (based on inverse distances). A notable algorithm is `nx.pagerank(G)`, which returns the PageRank of each node – “a ranking of the nodes based on the structure of incoming links” (originally for web pages importance) ⁵⁹. PageRank treats each directed edge as a “vote”, and the score can be interpreted as a node’s influence. In an exam context, they might ask for PageRank or centrality of nodes in a network (e.g., identify the most important node). These functions return dictionaries keyed by node with the centrality score.

- **Clustering coefficient:** `nx.clustering(G)` computes the clustering coefficient for each node, which measures how complete the neighborhood is (out of all possible edges between a node’s neighbors, the fraction that exist) ⁶⁰. The average clustering of the graph (`nx.average_clustering(G)`) is a single number often reported in network analysis.
- **Community detection:** NetworkX has some community algorithms. In the 2019 exam, for example, they used `nx.algorithms.community.kernighan_lin_bisection(G)` to partition the graph into two communities. This function attempts to find a near-balanced split of the graph that minimizes edge cuts between the two sets ⁶¹. Other community functions (like modularity-based detection) exist but may not have been used. If a question involves splitting the graph or finding groups, Kernighan-Lin or connected components could be the approach.
- **Network measures:** Functions like `nx.diameter(G)` compute the longest shortest-path in the graph (i.e., the diameter, which is the greatest distance between any two nodes) – although note that if the graph is disconnected, diameter is infinite (NetworkX will raise an error as seen in an example where a graph was not connected) ⁶². `nx.connected_components` we covered, and `nx.number_connected_components(G)` simply gives how many components there are.
- **Reading/Writing graphs:** If graph data is provided in a file format (GraphML, edge list, adjacency list, etc.), NetworkX has readers like `nx.read_graphml('file.graphml')` and writers like `nx.write_gml(G, 'file.gml')`. In the 2023 exam, for instance, they used `nx.read_graphml()` to load a graph ⁶³. If such a file is provided, this function would be necessary to import the graph structure.

When using NetworkX, it’s common to combine these functions. For example, one might build a graph from data (like co-authorship or adjacency matrix) and then **query**: *How many connected components does it have? What is the size of the largest component?* (use `nx.connected_components`). Or *Who are the top 5 nodes by PageRank or degree?* (use centrality functions or simply sort `G.degree`). Or *Compute the shortest path between two specific nodes* (use `nx.shortest_path`). Knowing these functions allows one to answer such questions.

Matplotlib (Pyplot) – Plotting Basics

Matplotlib is the foundational plotting library in Python, and **Pyplot** is its interface that provides a MATLAB-like set of functions for creating plots. We typically import it as `import matplotlib.pyplot as plt`. With Pyplot, you build a figure step by step by calling functions to plot data and adjust the appearance. Common plotting functions and their usage include:

- **Line plots:** `plt.plot(x, y)` plots a line (and/or markers) connecting the points defined by coordinate lists `x` and `y`. By default, `plt.plot` will draw a line through the points in order ⁶⁴. If you want just points without a line, you can add an argument like `plt.plot(x, y, 'o')` for circular markers. Multiple lines can be plotted by calling `plt.plot` multiple times or by providing multiple sets of `x,y` in one call.

- **Scatter plots:** `plt.scatter(x, y)` draws individual points (markers) at the given x,y coordinates without connecting them ⁶⁵. This is useful for visualizing relationships between two continuous variables (each point represents an observation). You can also pass an optional third argument to vary point sizes or use `c=` to vary colors.
- **Bar charts:** `plt.bar(categories, values)` creates a bar for each category with the given height. For horizontal bars, there's `plt.barh`. Typically, you provide a list of category names or indices and a list of corresponding values. This is useful for discrete/categorical data comparison.
- **Histograms:** `plt.hist(data, bins=...)` computes and plots a histogram of the data ⁶⁶. A histogram represents the distribution of a numeric variable by grouping data into bins and plotting the count (or density) of points in each bin. Matplotlib's `hist` uses `numpy.histogram` under the hood to get bin counts ⁶⁶. You can specify the number of bins or bin edges, and optionally set `density=True` to normalize to a probability density. This is a quick way to check distribution shape (normal, skewed, etc.).
- **Labels and titles:** After plotting data, it's important to label the axes and perhaps add a title. `plt.xlabel("X-axis label")` and `plt.ylabel("Y-axis label")` set the axis labels. `plt.title("Some Title")` adds a title to the plot. These functions make the plot self-explanatory.
- **Displaying or saving the figure:** `plt.show()` is used to display the plot in an interactive window (not always necessary in a notebook environment, where plots display automatically). If running as a script or if you want to force the figure to render at a specific point, `plt.show()` will do it. To save a plot to an image file, you can use `plt.savefig("filename.png")`.
- **Other plot types:** Matplotlib has a wide array of plotting functions. For example, `plt.scatter` we mentioned, `plt.boxplot(data)` for box-and-whisker plots, `plt.pie(values)` for pie charts, `plt.imshow(matrix)` to display an image or matrix data (useful for heatmaps if not using seaborn), etc. But in many data science use-cases, line, scatter, bar, and hist are among the most common. Error bars can be added with `plt.errorbar`, multiple subplots can be created with `plt.subplot` or `plt.subplots`, but these might be beyond the scope unless a question specifically needs multiple plots in one figure.

In practice, you might see code like:

```
plt.figure(figsize=(6,4))
plt.plot(years, counts, marker='o')
plt.title("Number of videos per year")
plt.xlabel("Year")
plt.ylabel("Count")
plt.grid(True)
plt.show()
```

This would create a nicely labeled line plot of counts over years with markers on points and a grid. The key takeaway is: **Matplotlib/Pyplot is imperative** – you issue commands to add elements to the current figure.

In an exam context, you may be asked to produce a specific plot (e.g., “Plot the number of videos per year using an appropriate plot type”). In such cases: - Decide the plot type (line plot for a trend over years, bar plot if years are treated as categories, etc. – here a line plot is suitable for a time trend). - Use the relevant

`plt.` functions to plot and label as required. - Ensure the plot is readable (labels, maybe a legend if multiple lines, etc.).

Remember that after plotting, the figure remains in memory; if you create multiple plots in succession, you might call `plt.figure()` to start a new one, or use subplots. But often, a simple scenario only needs one figure which `plt.show()` will display.

Seaborn – Statistical Data Visualization

Seaborn is a high-level visualization library built on Matplotlib, designed to make it easier to create attractive statistical graphics. It integrates well with pandas DataFrames, allowing you to specify semantic mappings (like color by group) directly. With seaborn (imported as `sns`), some common functions that might come up include:

- **Scatter plots with grouping:** `sns.scatterplot(x="var1", y="var2", hue="category", style="category", data=df)` will plot a scatter of var1 vs var2 and distinguish points by color (and/or marker style) according to the categorical variable ⁶⁷. Seaborn's scatterplot is similar to `plt.scatter`, but it adds the convenience of directly using DataFrame column names and adding semantics like hue (color) for a third dimension. For example, you could easily plot and visually separate two classes in the data by color. The documentation notes that *"the relationship between x and y can be shown for different subsets of the data using the hue, size, and style parameters"* ⁶⁷.
- **Distribution plots:** Seaborn has several functions to plot distributions:
 - `sns.histplot(data=df, x="variable", hue="group")` will plot a histogram of the given variable, and if a *hue* is provided, it will produce multiple histograms (layered or side by side) colored by the group ⁶⁸. This is great for comparing distributions between categories. By default, `histplot` will show counts, but it has options for showing densities, cumulative distributions, etc.
 - `sns.kdeplot(...)` plots a kernel density estimate (smoothed curve) of a distribution. In practice, `sns.histplot(..., kde=True)` can overlay a KDE on the histogram.
 - `sns.boxplot(x="category", y="value", data=df)` creates box plots for the distribution of *value* within each category. This is useful to compare medians and spread across groups.
 - `sns.violinplot` is similar to boxplot but shows a full density estimate on each side.
 - `sns.pairplot(df)` will generate a grid of scatter plots for each pair of numerical variables (and histograms on the diagonal), which is useful for exploratory data analysis (though potentially too large for an exam setting unless a small number of variables).
- **Bar plots and count plots:** `sns.barplot(x="category", y="value", data=df, estimator=np.mean)` will show the mean (default estimator is mean) of *value* for each category with error bars (confidence interval). This is good for summarizing comparisons with uncertainty. If you just want to plot counts of observations in each category, `sns.countplot(x="category", data=df)` will display a bar for each category count.
- **Heatmaps:** `sns.heatmap(data_matrix)` will display a matrix (like a correlation matrix or adjacency matrix) as color-coded cells. Often you'll see `sns.heatmap(corr_matrix, annot=True)` to show a correlation matrix with numeric values annotated. Heatmaps are useful for visualizing matrices or clusters (e.g., in social network adjacency or confusion matrices for classification results).
- **Faceting and grids:** Seaborn has the ability to create multiple plots by subgroup via `FacetGrid` or high-level functions like `sns.relplot` and `sns.displot` with a `col` or `row` parameter. For

instance, `sns.relplot(x="var1", y="var2", col="category", data=df)` would create a separate scatter plot for each category value, laid out in columns. This may or may not come up, but it's good to know if you need to compare patterns across subsets.

Seaborn generally produces more visually appealing plots with less code. For example, if the exam dataset has a time series with a trend and a seasonal pattern, they might expect a line plot by year and perhaps a boxplot by month to see seasonality. One could use `sns.lineplot(x="Year", y="value", data=df)` for the trend and `sns.boxplot(x="Month", y="value", data=df)` for seasonal distribution per month.

One important note: **Seaborn sets default styles** (via `sns.set_theme()` usually) that make plots look nicer (soft colors, grid in background, etc.). If consistent styling is desired, one might call `sns.set_style("whitegrid")` for instance, which adds a white background with gridlines – commonly used for statistical plots.

In summary, **Seaborn** functions simplify many plotting tasks: - Use them when you want to map data columns to visual elements (like color = category). - They often combine what would be multiple matplotlib calls into one. For example, `sns.histplot(hue="species", data=tips, x="total_bill")` internally will loop over species and color the histogram, whereas with pure matplotlib you'd manually filter the DataFrame and plot histograms for each species.

Finally, since Matplotlib underlies Seaborn, you can still use `plt.title`, `plt.xlabel`, etc., after a seaborn plot to adjust labels, or use seaborn's own functionality (many seaborn functions return an Axes object which you can modify).

These are the general functions and methods across the libraries that are most likely to come up given their widespread utility in data analysis tasks. By being familiar with them, one can handle tasks such as data cleaning and transformation (NumPy, pandas), statistical testing (SciPy), modeling (statsmodels, scikit-learn), and result visualization (Matplotlib, Seaborn) – covering the end-to-end workflow of many data science problems. Each function mentioned here addresses a common requirement, and together they form a powerful toolkit for analysis.

1 2 3 4 5 6 7 8 9 10 Useful NumPy functions in Data Science | by Akriti Upadhyay | Medium

<https://medium.com/@akriti.upadhyay/useful-numpy-functions-in-data-science-dc078f7cd36b>

11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 20 pandas Functions to complete 80% of your Data Science tasks | by Deepak Sharma | Medium

<https://medium.com/@deepaksharma2494/20-pandas-functions-to-complete-80-of-your-data-science-tasks-cefa535a629e>

29 30 31 33 Statistical functions (scipy.stats) — SciPy v1.16.2 Manual

<https://docs.scipy.org/doc/scipy/reference/stats.html>

32 SciPy Statistical Significance Tests

https://www.w3schools.com/python/scipy/scipy_statistical_significance_tests.php

34 35 chisquare — SciPy v1.16.1 Manual

<https://docs.scipy.org/doc/scipy-1.16.1/reference/generated/scipy.stats.chisquare.html>

36 37 pearsonr — SciPy v1.16.2 Manual

<https://docs.scipy.org/doc/scipy-1.16.2/reference/generated/scipy.stats.pearsonr.html>

38 41 Interpreting Linear Regression Through statsmodels .summary() | by Tim McAleer | The Startup | Medium

<https://medium.com/swlh/interpreting-linear-regression-through-statsmodels-summary-4796d359035a>

39 40 Fitting models using R-style formulas - statsmodels 0.14.6

https://www.statsmodels.org/stable/example_formulas.html

42 train_test_split — scikit-learn 1.8.0 documentation

https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

43 TfidfVectorizer — scikit-learn 1.8.0 documentation

https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

44 exam_2024.ipynb

<file:///file-MFEWBHtfSY5qMS4aBNuSVW>

45 46 TruncatedSVD — scikit-learn 1.8.0 documentation

<https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.TruncatedSVD.html>

47 48 Python Machine Learning - Logistic Regression

https://www.w3schools.com/python/python_ml_logistic_regression.asp

49 50 accuracy_score — scikit-learn 1.8.0 documentation

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.accuracy_score.html

51 52 53 54 55 58 60 Tutorial — NetworkX 3.6.1 documentation

<https://networkx.org/documentation/stable/tutorial.html>

56 57 connected_components — NetworkX 3.6.1 documentation

https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.components.connected_components.html

59 pagerank — NetworkX 3.6.1 documentation

https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.link_analysis.pagerank_alg.pagerank.html

61 62 exam_2019.ipynb

<file:///file-CrGGwhPHP8LNhammXYvTFc>

63 exam_2023.ipynb

file:///file-Qrs8z9j7GpuELUSpgmi3Jm

64 Matplotlib Plotting - W3Schools

https://www.w3schools.com/python/matplotlib_plotting.asp

65 Matplotlib Scatter - W3Schools

https://www.w3schools.com/python/matplotlib_scatter.asp

66 matplotlib.pyplot.hist

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.hist.html

67 seaborn.scatterplot — seaborn 0.13.2 documentation

<https://seaborn.pydata.org/generated/seaborn.scatterplot.html>

68 seaborn.histplot — seaborn 0.13.2 documentation

<https://seaborn.pydata.org/generated/seaborn.histplot.html>